

[K] Using a Convolutional Neural Network To Simulate Fabric

Hugo Nilsson

Chalmers University of Technology

(Dated: May 31, 2026)

Soft bodies are some of the most complicated and computationally intensive objects to simulate. For this reason machine learning models can be used to improve performance of while preserving a reasonable degree of accuracy. In this project it was investigated if a Convolutional Neural Network (CNN) can be used to model cloth dynamics. This meant the program was simple to implement using pytorch but with the drawback that there is very little control over the shape of the fabric as opposed to other common models, like Graph Neural Networks (GNN).

Here an 11 layer residual CNN with around 0,64M trainable parameters was successfully trained to simulate a 20 by 20 vertex fabric that remained stable under light disturbances. The model was trained using a dataset produced by a simple semi-implicit Euler simulator. This shows that CNNs can serve as a practical alternative to more complex architectures when simulating soft bodies.

Project Topic: Prediction of cloth motion in a spring-mass particle system

Teaching Assistant: John Tember

I. INTRODUCTION

Today there are numerous fields in which large particle simulations are used to predict bulk properties, large molecules, fluid simulations or as will be discussed here, soft body simulations. In traditional cloth simulations, the fabric is often represented by a collection of particles connected by springs and dampers [1], but these simulations are however generally quite expensive to both create and run and are thus often hard to scale [2]. This makes it attractive to use Machine Learning (ML) to make computations significantly faster while maintaining a reasonable amount of accuracy. Examples include Machine Learned Potentials (MLP) in material science used to calculate bulk properties [3] or Graph Neural Networks (GNN) used for cloth simulations [4].

As mentioned, modern ML cloth simulations generally uses GNNs which explicitly control particle connections and thus fabric shape, this project however will investigate if a Convolutional Neural Network (CNN) can be trained to simulate a square fabric. A CNN would offer a more straight forward architecture, simple notation and easy visualization, making it an interesting baseline to investigate, despite limitations.

II. OVERVIEW

Pure CNN. A simple architecture for simulating a particle system is to just hand a neural network a set of point positions paired with their positions in the next timestep and train it to predict the next position to any input position as seen in figure 1a. This model can be summarized by equation 1.

$$\vec{x}_t \rightarrow \vec{x}_{t+1} \quad (1)$$

The upside of this is that the Machine Learning integrator is very simple to create and understand, while the downside is that this means the network needs to learn

everything at once, it has no notion of velocities, forces or previous states. This means that it is very sensitive to errors as a small error in one timestep will only grow larger in the next.

Another significant upside of this method is that since it is only reliant on position data, this can easily be trained on motion capture data instead of a sophisticated simulation.

Force-Field Predicting CNN. While the idea of the pure CNN model is to predict everything on its own, a force-field predicting CNN is meant to learn only what is necessary and evaluate the rest using a conventional physics integrator. The reasoning behind this is that it would allow the model to be smaller and easier to train as there is less to learn, while it still deals with the complicated physics calculations. The visualized pipeline can be seen in figure 1b. This also allows some freedom when using the model as the training is independent of some physical parameters such as the timestep and particle mass, this means that the model can be trained on a dataset with very stable parameters with a short timestep and high mass and then adjust these when running the simulation. Additionally the forces can be scaled to create stronger or weaker springs, this does however also influence gravity in the current setup, but it is of course also possible to train the model on a weightless dataset and add gravity during integration. Equations 2 and 3 describe the full pipeline.

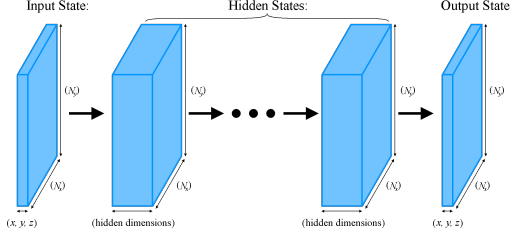
$$\{\vec{x}_t, \vec{v}_t\} \rightarrow \vec{F}_t \quad (2)$$

Then using an integrator such as semi-implicit Euler in this case:

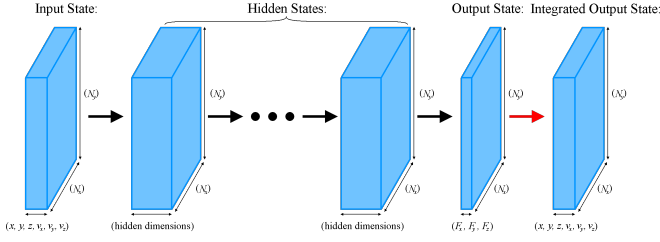
$$\vec{a}_t = \frac{\vec{F}_t}{m}, \quad \vec{v}_{t+1} = \vec{v}_t + \vec{a}_t \Delta t, \quad \vec{x}_{t+1} = \vec{x}_t + \vec{v}_{t+1} \Delta t. \quad (3)$$

Force-Field Predicting CNN with Residual Blocks. A known issue with deep Convolutional Neural Networks is that many complex tasks benefit from very

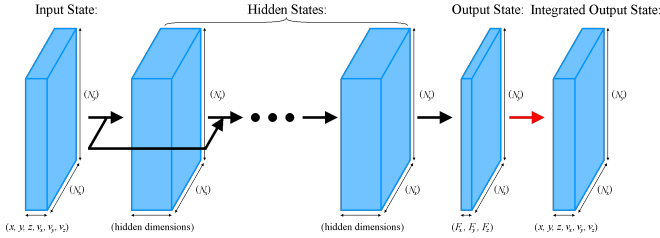
deep networks, but at the same time, that depth makes them harder to train as gradients tend to either vanish or explode [5]. The solution to this is to allow some signals to pass some of the convolutional layers, often referred to as a residual network [5]. This architecture, visualized in figure 1c, is essentially the same as the Force-Field Predicting CNN, but with the ability to have more layers, which means the model could be able to capture more details in the dynamics, at the cost of more expensive computations.



(a) **Pure CNN.** A CNN uses current positions to predict the next set of positions.



(b) **Force-Field Predicting CNN.** A CNN uses positions and velocities to calculate forces that are used to integrate new positions and velocities (as seen in red).



(c) **Force-Field Predicting CNN with Residual Blocks.** Skip connections allows a larger number of layers to be used.

FIG. 1: **Model architectures.** Here N_x and N_y refer to number of vertices in each direction, and (x, y, z) denote spatial coordinates.

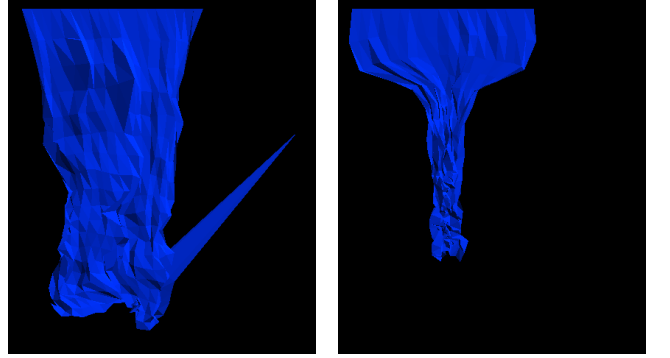
In table I the three architectures are compared, the initial decision was to use the Force-Field Predicting CNN, and if the problem of vanishing/exploding gradients appears residual blocks can be added.

III. METHOD

The first step in the project was to create a basic simulation that was able to generate datasets for the model to use. The resulting simulation was a Provot-style spring-

mass-damper system integrated using a semi-implicit Euler integrator [1].

Dataset generation. The simulation was run for approximately 5 000 000 timesteps where every 50th timestep was first allowed to be manipulated through having a single point dragged away in a random direction, or all points compressed toward a common center as seen in figure 2. Then positions, velocities and forces were recorded for that timestep. Saving only a fraction of the timesteps increased the diversity of the dataset as the consecutive timesteps are often very similar. This strategy allowed datasets of sizes around 5 GB rather than beyond 50 GB if all timesteps were saved.



(a) Single particle dragged away (b) Entire fabric compressed during dataset creation.

FIG. 2: **Mass-Spring-Damper System used to generate datasets.** During dataset generation the fabric was subjected to disturbances like compression and dragging.

Inspecting the dataset. A program was created that allows loading positions from the dataset instead of calculating them. This allowed effective visual inspection of datasets.

Loading dataset. In order to make it possible to load very large datasets beyond the capacity of the Random Access Memory (RAM) and Video Random Access Memory (VRAM), a dataset chunker was constructed which allowed the dataset to be loaded in chunks of equal size and then trained consecutively. Out of each chunk 80 % of the samples were shuffled and used for training while the remaining 20 % were used for validation and were not shuffled. The batch size could then be adjusted to accurately use the desired amount of VRAM.

A normalisation step was implemented, but due to an indexing error it was only applied to the forces and had no practical effect on positions and velocities in the dataset.

Training. The chosen model was a CNN with a total of roughly 0.64 M trainable parameters, 11 convolutional layers, three skip connections, and dilation, as seen in the schematic in figure 3. Directly after each convolutional layer, Batch Normalisation (BN) was used before the Gaussian Error Linear Unit (GELU) activation function was applied.

TABLE I: **Considered model architectures.** The following model architectures were considered for the project.

Simulation method	Use case scenario	Features	Suitability
Pure CNN	Could be useful for very simple simulations	Simplicity, it lets the network handle everything	Could be possible, but may very well be too basic
Force-Field Predicting CNN	The next step up from a pure CNN, should be able to handle more complex simulations	The particle system is based in physics, the network only has to learn forces	Could be useful the major limitation is if network has to be very deep
Force-Field Predicting CNN with Residual Blocks	The same as Force-Field Predicting CNN but should be able to handle even more complex simulations	The same as Force-Field Predicting CNN but more powerful	Can be built from Force-Field Predicting CNN if needed

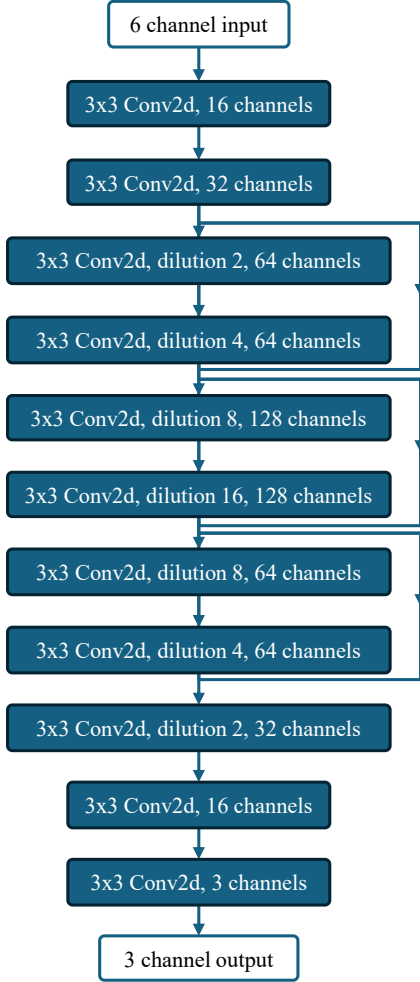


FIG. 3: **Chosen structure.** The chosen structure was an 11 layer CNN with residuals and dilation.

The chosen loss function was the Mean Squared Error (MSE) of predicted forces to target forces and Adam was used as the optimiser. The full set of training parameters are listed in table II.

TABLE II: **Training parameters.** The following training parameters were used in the project.

Parameter:	Value:
Epochs:	1 000
Learning Rate:	0,0001
Batch Size:	2 048
Number of Samples:	163 840
Padding Mode:	zeros

Running the simulation. When running the simulation, the position tensor and velocity tensor are concatenated to create a $(6, N_x, N_y)$ tensor which is sent to the CNN which returns a tensor containing all forces in that timestep, the forces are then integrated using the update equations (equations 2-3). Finally velocities are clipped to prevent instability.

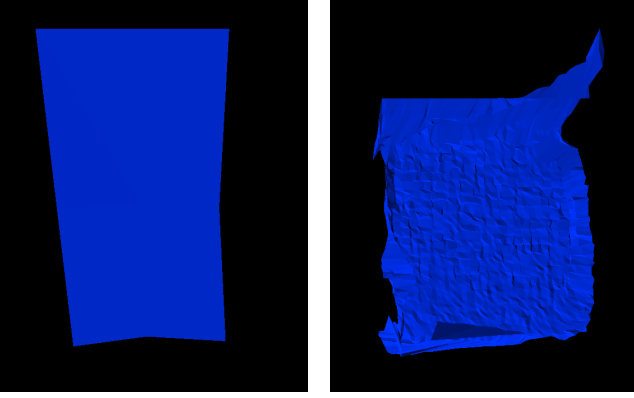
The simulation is visualised with OpenGL using GLFW.

Training hardware. A single RTX 5070 with 12 GB VRAM was used for all training and the final run took around 6,0 h.

IV. RESULTS AND DISCUSSION

Early tests. Early tests indicated that small models could be trained to stably simulate tiny samples of size $N_x = 3, N_y = 3$ as seen in figure 4a, this was seen as a good indication that the project would work. When samples were scaled up however the model could not achieve stability as it seemed to collapse from the edges (see figure 4b). The reason this happened was assumed to be because the fabric structure is fundamentally different along the edges from the internal vertices where the edges had some neighbours with length 0. In an attempt to prevent this reflective padding was tested, but with no significant improvement.

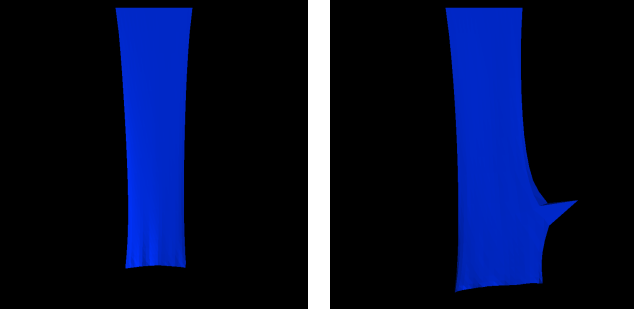
Final simulation. The final model was tested on a 20 by 20 vertex fabric and was very stable in an undisturbed state and allowed some light deformation (see figures 5a-5b). If too much tension was added to the system for example by manually displacing a single vertex too far,



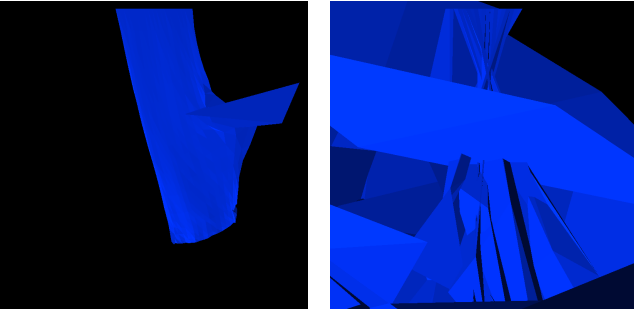
(a) Tiny sample with 3 by 3 vertices. (b) Early test with 50 by 50 vertices.

FIG. 4: Early tests of different size. Early tests were somewhat effective for tiny samples but failed when scaled up.

the neighbouring vertices quickly gained velocity and the fabric soon disintegrated (see figures 5c-5d).



(a) **Stable when undisturbed.** When undisturbed, the fabric relaxes to a stable drape. (b) **Allows light manipulation.** If a particle is pulled away lightly, stability is retained.



(c) **Too much tension creates instability.** If a particle is pulled too hard, the surrounding particles quickly became unstable. (d) **Instability spreads through the fabric.** If part of the fabric is moving too fast, the instability spreads to the entire fabric.

FIG. 5: Model architectures. Here N_x and N_y refer to number of vertices in each direction, and (x, y, z) denote spatial coordinates.

Training dynamics. As seen in figure 6, the training and validation losses were both smooth, continually decreasing and the validation loss was consistently lower than the training loss. This is taken as a sign that the model and thus an indication that increasing the model size would likely be beneficial.

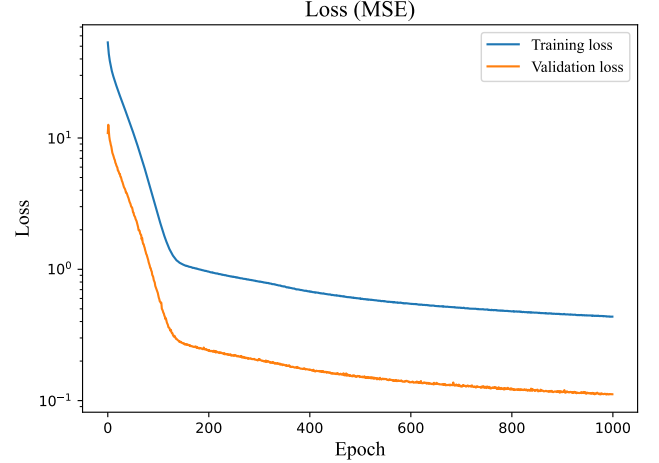


FIG. 6: Training and validation loss. The loss curves where both smooth and decreasing, the validation loss is constantly lower than training loss. The final training loss was 0,436 and the final validation loss was 0,112

Discussion. This project successfully demonstrated that a CNN can produce stable predictions to cloth dynamics under small disturbances. Additionally, since the validation error is consistently lower than training error the final model could be improved by increasing training time and model size. This means that a CNN can be seen as a viable alternative to other architectures especially when the fabric is rectangular. However, since the final model used the "zeros" padding mode it may be possible to extend applications to non rectangular fabrics, although this was not tested.

Limitations. An expected limitation was the structure of the fabric since CNNs are naturally suitable to rectangular fabrics, early tests indicated that small CNNs struggle even with rectangular fabrics as even within a square there are significant variations in connections. Since this limitation was overcome by simply adding depth to the network however, it may be possible to overcome other shape limitations in the same way, as discussed previously.

This project was limited to simple simulated datasets and thus the model was constrained to the same realism as the basic Euler integrator that was constructed for the task of generating datasets. Moreover, a model trained on a specific fabric size only worked to simulate fabrics of a very similar size, to large or small and the simulation instantly loses stability.

During the very final stages of this project a very serious bug was found that revealed that the network had

silently not trained since a very early version of the code. Because of this a large part of the project was aimed toward getting the model to learn anything at all, meaning that when the code was fixed and the model did work, there was no time to carry out significant tests and variations.

V. CONCLUSIONS AND OUTLOOK

Conclusion. This project shows that a Convolutional Neural Network (CNN) can be used to simulate fabrics using a force-field predicting scheme. It also shows that the developed model is still underfitting and has thus not yet reached its full potential. This means that CNNs can be used to simulate this kind of simple physical system. The main limitation of this scheme is that the model is limited in realism to the degree of the simulator used to produce the dataset, this is since the model is training on forces which would be not be possible to measure experimentally to the required degree, as opposed to position or possibly velocity predicting schemes.

Outlook. During development of this model it has become clear that there are some significant improvements that can be made. Firstly, with a physics informed loss function that for example evaluate the energies of the system could train the model to never increase the energy of the system, potentially enforcing stability. Secondly, since the padding mode "zeros" allowed the model to learn to ignore some connections, it might be possible to train the model to treat "zero connections" as "not connections", making it possible to simulate virtually any

shape. When it comes to shapes of the fabric, the trained model was also limited to shapes similar to the shape it was trained on, this means that if a dataset is constructed of fabrics of many different shapes and sizes, a more general model could be created.

Finally, it may be superior to use an architecture that takes positions as input, calculates velocity by comparing each position with the next and then allowing the model to predict velocity changes from that. This may allow the model to be trained on motion capture data for example.

VI. CONTRIBUTIONS

Hugo Nilsson created all figures, wrote this report, and the code, in its entirety. Microsoft Copilot and Ecoisa AI has been used for explaining concepts, finding bugs etc. but everything from code to report has been composed and written by me.

VII. CONFLICT OF INTEREST

The author does not have a conflict of interest.

VIII. DATA AND CODE AVAILABILITY

The simulation, training and dataset generation code are made publicly available at https://github.com/SomeCucumber/ML_fabric_simulation.

The visualisation stack is available at <https://github.com/SomeCucumber/OpenGL>.

-
- [1] X. Provot, Deformation constraints in a mass-spring model to describe rigid cloth behavior, *Graphics Interface* **23(19)** (2001), describes a traditional fabric simulation setup.
 - [2] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia, Learning to simulate complex physics with graph networks, *37th International Conference on Machine Learning, ICML 2020 Part F168147-11*, 8428 (2020), describes a state of the art machine learned simulator and some of its usecases.
 - [3] E. Kocer, T. W. Ko, and J. Behler, Neural network potentials: A concise overview of methods, *Annual Review of Physical Chemistry* **73**, 163 (2022), machine learning use in chemistry and material science.
 - [4] Y. Li, X. Wang, X. Song, C. Fan, B. Zhu, J. Peng, Q. Liang, J. Lam, K. Y. Sze, and K. W. Kwok, Graph-neuralcloth: A graph-neural-network-based framework for non-skinning cloth simulation, *Advanced Intelligent Systems* **8**, e202501120 (2026), a more niche model used for simulating cloth specifically.
 - [5] K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition* **2016-December**, 770 (2015), explains the use of residuals in a CNN.